Project Executable Files

Date	30 September 2026
Team ID	6ab3ec14b5170a9b58b3a658
Project Name	Implement Client Script and UI Policy
Maximum Marks	3 Marks

Project Executable Files

This section requires submission of all executable and deployable files for your project. Ensure all files are properly organized, named, and uploaded to the specified submission platform. The evaluator must be able to run or deploy your solution using the provided files and instructions.

Step 1: Submission Checklist

S.No	Item to Submit	Submitted (Yes / No / NA)
1	Complete source code (all files and folders)	Yes – 3 Client Scripts + UI Policy configuration (Appendix A.2 of Phase 7 Documentation)
2	README / Setup Guide (instructions to run the project)	Yes – Step 4 below and Phase 7 Documentation (Ch. 5 & 6)
3	requirements.txt / package.json / dependency file	NA – configuration-based ServiceNow project; no external dependencies
4	Database schema / seed files (if applicable)	NA – uses the out-of-the-box Incident table
5	Environment configuration file (.env.example or similar)	NA
6	Deployed application URL (if hosted)	Yes – ServiceNow instance (see Step 3)
7	APK / Executable binary (if applicable for mobile/desktop apps)	NA
8	Dockerfile / Containerization config (if applicable)	NA
9	Test files and test results	Yes – Test activities and results in Ch. 7 & 8
10	Demo video	Yes

Step 2: File / Folder Structure

Provide an overview of your project's file and folder structure below:

```
/ServiceNow-Instance (Incident table configuration)
|-- System UI > UI Policies
|    `-- High Impact Control (Incident | Impact is 1 - High)
|         |-- Action: Assignment group - Mandatory
|         `-- Action: Urgency - Read-only
`-- System UI > Client Scripts
     |-- Auto set urgency for high impact   (onChange - Impact)
     |-- Prevent save if Assigned To missing (onSubmit)
     `-- Prevent state change via list edit  (onCellEdit - State)

/docs
|-- Phase_7_Documentation.docx    - Documentation, tests, scripts
`-- Project_Executable_Files.docx - This document
```

Step 3: Deployment / Access Details

Field	**Details**
Hosted / Deployed URL	https://developer.servicenow.com/dev.do
Login Credentials (Demo)	karthikavaithi0608@gmail.com
Platform / Hosting Provider	ServiceNow (cloud-hosted instance)
Repository Link	https://github.com/Vaitheeswari-22/Implement-Client-Script-UI-Policy-Incident-
Demo Video Link	https://drive.google.com/file/d/1evcrPDzm7WxjhnfUt1y1bPkmOgJjBpgm/view?usp=sharing

Step 4: Run Instructions

Provide clear, step-by-step instructions for the evaluator to run your project locally:

Pre-requisites:

- A ServiceNow instance with administrative or configuration access (System UI > UI Policies, System UI > Client Scripts, Incident table).
- Modern web browser (Chrome, Edge or Firefox) with a stable internet connection; minimum 4 GB RAM (8 GB recommended).
- Incident fields available: Impact, Urgency, Assignment group, Assigned to, State.

Step 1:

Log in to the ServiceNow instance with an admin / configuration role.

Step 2:

Go to System UI > UI Policies > New. Name: High Impact Control; Table: Incident; Active: true; Condition: Impact is 1 - High; Reverse if false: true. Click Submit.

Step 3:

In the UI Policy Actions related list, click New: Assignment group > Mandatory: true, Submit. Click New again: Urgency > Read-only: true (Visible unchanged), Submit.

Step 4:

Go to System UI > Client Scripts > New. Name: Auto set urgency for high impact; Table: Incident; Type: onChange; Field name: Impact; Active: true. Paste the onChange script from Appendix A.2 and Submit.

Step 5:

New Client Script. Name: Prevent save if Assigned To missing; Table: Incident; Type: onSubmit; Active: true. Paste the onSubmit script and Submit.

Step 6:

New Client Script. Name: Prevent state change via list edit; Table: Incident; Type: onCellEdit; Field name: State; Active: true. Paste the onCellEdit script and Submit.

Step 7:

Go to Incident > Create New. Set Impact to High and leave Assigned to empty, then Submit. Expected: save is blocked with an Assigned To error; Urgency is set to High and read-only; Assignment group is mandatory.

Step 8:

Select a user in Assigned to and Submit. Expected: the Incident saves successfully.

Step 9:

Open an existing High Impact Incident and change Impact to Medium. Expected: the mandatory / read-only rules reverse and the record saves.

Step 10:

Go to Incident > All and double-click the State cell of any Incident. Expected: an alert says State cannot be updated using list editing, and the value is unchanged.

Step 5: Known Issues / Limitations

S.No	Known Issue / Limitation	Workaround / Status
1	Validations are client-side only: they run in the browser form/list UI, so updates made through imports, REST APIs or server-side scripts are not covered.	Planned enhancement: add a server-side Business Rule / Data Policy for the same rules.
2	The onCellEdit script blocks every State edit from the list view for all users, using a basic alert, and blocked attempts are not logged.	Open the Incident form to change State. Planned: descriptive inline message and audit logging (Ch. 10).
3	Behaviour depends on out-of-the-box field names (impact, urgency, assigned_to, state), a single active UI Policy on these fields, and Reverse if false being checked.	Verify backend field names and retest after any change to the UI Policy or scripts (Ch. 9).